INSTITUTO SUPERIOR TECNOLÓGICO SUDAMERICANO

Carrera de Desarrollo de Software

SINKA

Plataforma web de concentración colaborativa en línea con emparejamiento aleatorio

MANUAL DE PROGRAMADOR

Autor: John David Calle Hernández

Director: Ing. David Paúl Rosales Herrera, Mgs.

Loja - Ecuador

2026

Tabla de contenido

(Haga clic derecho sobre el índice y seleccione "Actualizar campos" en Microsoft Word para generar los números de página.)

1. Introducción 4

1.1. Propósito del Manual 4

1.2. A Quién Está Dirigido 4

1.3. Requisitos Previos 4

2. Visión General del Sistema 4

2.1. Arquitectura General 4

2.2. Entornos de Despliegue 5

3. Stack Tecnológico Completo 5

3.1. Backend 5

3.2. Frontend 5

3.3. Infraestructura 6

4. Entorno de Desarrollo 6

4.1. Instalación Local del Backend 6

4.2. Instalación Local del Frontend 7

5. Arquitectura del Backend 8

5.1. Estructura de Carpetas 8

5.2. El Patrón de Cuatro Capas por Módulo 8

5.2.1. Capa de catálogo/configuración fija — payments/catalog.py 8

5.2.2. Capa de modelo — payments/models.py 9

5.2.3. Capa de servicio — payments/service.py 9

5.2.4. Capa de API — payments/api/router.py 10

5.3. Segundo Patrón de Ejemplo: gamification 11

5.4. El Bus de Eventos Interno (EventBus) 11

5.5. Autenticación de WebSocket 12

5.6. Configuración y Conexión a la Base de Datos 13

6. Arquitectura del Frontend 14

6.1. Estructura de Carpetas 14

6.2. Manejo de Estado con Zustand 14

6.3. El Cliente de API (lib/api.ts) 15

6.4. Internacionalización (i18n) 16

6.5. Tema Claro/Oscuro (ThemeProvider) 16

6.6. Ejemplo de Página: Dashboard 16

7. Base de Datos 17

7.1. Tablas Principales 17

7.2. Migraciones con Alembic 17
7.2.1. Ejemplo: migración 0001 (tablas base) 18
7.2.2. Ejemplo: migración 0022 (datos semilla) 18
8. Módulos del Sistema 19
8.1. identity — Identidad y Autenticación 19
8.2. matchmaking — Emparejamiento 19
8.3. sessions — Sesiones de Enfoque 19
8.4. gamification — XP, Nivel, Racha y Tienda 19
8.5. social — Vínculos y Jardín 20
8.6. groups — Grupos 20
8.7. scheduling — Citas 20
8.8. achievements — Logros 20
8.9. payments — Pagos 20
8.10. reports — Buzón 20
8.11. announcements — Anuncios 21
8.12. admin — Panel de Administración 21
9. API REST 21
9.1. Formato de Respuesta y Autenticación 21
9.2. Endpoints Principales por Módulo 21
10. WebSockets 22
10.1. El Canal de la Sesión 22
10.2. Señalización WebRTC 22
11. Despliegue 22
11.1. Backend en Railway 22
11.2. Frontend en Vercel 23
11.3. Webhook de Stripe en Producción 23
12. Extensión y Mantenimiento 23
12.1. Cómo Agregar una Funcionalidad Nueva 23
12.2. Ejecutar la Suite de Pruebas 24
13. Buenas Prácticas del Proyecto 25

1. Introducción

1.1. Propósito del Manual

Este manual técnico documenta la arquitectura, el stack tecnológico, la organización del código y los procedimientos de instalación, extensión y despliegue del sistema SINKA. Está pensado para que cualquier programador —incluyendo quien no participó en el desarrollo original— pueda entender cómo está construida la plataforma, levantar un entorno de desarrollo local, modificar un módulo existente o añadir uno nuevo siguiendo el mismo patrón arquitectónico, y desplegar los cambios a producción.

1.2. A Quién Está Dirigido

Este manual está dirigido a desarrolladores de software con conocimientos de Python, TypeScript/React y bases de datos relacionales. No es necesario haber trabajado antes con FastAPI o Next.js específicamente, pero sí se asume comodidad general con APIs REST, SQL y control de versiones con Git.

1.3. Requisitos Previos

- Python 3.11 o superior.
- Node.js 18 o superior y npm.
- Una base de datos PostgreSQL accesible (en producción, SINKA usa Supabase).
- Una instancia de Redis accesible (en producción, SINKA usa Upstash).
- Git, y una cuenta en GitHub con acceso al repositorio del proyecto.
- Opcional: una cuenta de Stripe en modo de prueba, solo si se va a trabajar en el módulo de pagos.

2. Visión General del Sistema

2.1. Arquitectura General

SINKA sigue una arquitectura cliente-servidor desacoplada: un frontend Next.js que se comunica por HTTP/REST y WebSocket con un backend FastAPI, el cual persiste su estado en PostgreSQL y usa Redis como caché y como mecanismo auxiliar. El backend está organizado como un "monolito modular": una única aplicación FastAPI, pero internamente dividida en módulos de dominio independientes que se comunican entre sí mediante un bus de eventos interno (EventBus), evitando que los módulos se importen directamente entre ellos para operaciones que no son su responsabilidad principal.

La comunicación en tiempo real (temporizador Pomodoro compartido y señalización de video) se resuelve con WebSocket nativo de FastAPI más WebRTC punto a punto entre los dos navegadores de los usuarios emparejados: SINKA no tiene un servidor de medios centralizado, solo retransmite los mensajes de señalización (SDP/ICE) necesarios para que los navegadores establezcan la conexión de video directamente entre sí.

Componente	Tecnología	Responsabilidad
Frontend	Next.js 14 (App Router) + TypeScript	Interfaz de usuario, estado de cliente, consumo de la API
Backend	FastAPI (Python 3.11) + SQLAlchemy async	Lógica de negocio, autenticación, API REST y WebSocket
Base de datos	PostgreSQL (Supabase)	Persistencia de usuarios, sesiones, gamificación, etc.
Caché / auxiliar	Redis (Upstash)	Caché y soporte a funciones que requieren baja latencia
Tiempo real	WebSocket (FastAPI) + WebRTC (navegador a navegador)	Temporizador compartido, chat de sesión y señalización de video
Pagos	Stripe Checkout + Webhooks	Compra de paquetes de FocusCoins con dinero real

2.2. Entornos de Despliegue

Entorno	Plataforma	URL
Frontend (producción)	Vercel	https://sinka-eight.vercel.app
Backend (producción)	Railway	https://sinka-production-f624.up.railway.app/api
Base de datos	Supabase (PostgreSQL gestionado)	—
Redis	Upstash (Redis gestionado, protocolo TLS)	—

3. Stack Tecnológico Completo

3.1. Backend

- FastAPI — framework web asíncrono para la API REST y WebSocket.
- SQLAlchemy 2.x (modo async) — ORM, con sesiones AsyncSession y motor create_async_engine.
- Alembic — migraciones de esquema versionadas (22 migraciones al momento de este manual).
- Pydantic / pydantic-settings — validación de datos y carga tipada de variables de entorno.
- asyncpg — driver asíncrono de PostgreSQL usado por SQLAlchemy.
- bcrypt — cifrado de contraseñas.
- PyJWT — emisión y verificación de tokens de acceso y refresco.
- stripe (SDK oficial de Python) — Checkout Sessions y verificación de webhooks.
- uvicorn — servidor ASGI usado tanto en desarrollo como en producción (Railway).
- pytest + pytest-asyncio — pruebas unitarias.

3.2. Frontend

- Next.js 14 (App Router) — enrutamiento basado en archivos, Server y Client Components.

- React 18 + TypeScript.
- Tailwind CSS — utilidades de estilo, combinadas con un sistema de tokens propio (frontend/src/lib/theme.ts).
- Zustand — manejo de estado de cliente (por ejemplo, el store de autenticación).
- framer-motion — animaciones de interfaz.
- lucide-react — íconos SVG.
- simple-peer — abstracción sobre WebRTC para la videollamada punto a punto.

3.3. Infraestructura

- Railway — hospedaje del backend (contenedor construido con Nixpacks).
- Vercel — hospedaje y build del frontend.
- Supabase — PostgreSQL gestionado, con pooler de conexiones.
- Upstash — Redis gestionado, accesible por TLS (rediss://).
- Stripe — procesamiento de pagos con tarjeta.

4. Entorno de Desarrollo

4.1. Instalación Local del Backend

Paso 1: Crear y activar un entorno virtual

```
cd backend
python -m venv venv
.\venv\Scripts\Activate.ps1   # Windows / PowerShell
# source venv/bin/activate     # Linux / macOS
```

Paso 2: Instalar dependencias

```
pip install -r requirements.txt
```

Paso 3: Configurar variables de entorno

Copie backend/.env.example a backend/.env y complete los valores reales. Nunca se debe subir el archivo .env al repositorio.

```
# PostgreSQL (Supabase)
DATABASE_URL=postgresql+asyncpg://
postgres:<password>@db.<proyecto>.supabase.co:5432/postgres

# Redis (Upstash)
REDIS_URL=rediss://default:<token>@<host>.upstash.io:6379

# JWT
JWT_SECRET=cambia_esto_por_una_cadena_aleatoria_larga_minimo_32_chars
JWT_ALGORITHM=HS256
JWT_ACCESS_TOKEN_EXPIRE_MINUTES=30
JWT_REFRESH_TOKEN_EXPIRE_DAYS=7

# App
APP_ENV=development
CORS_ORIGINS=http://localhost:3000
```

```
# Stripe (modo prueba: claves sk_test_ / pk_test_)
STRIPE_SECRET_KEY=sk_test_...
STRIPE_PUBLISHABLE_KEY=pk_test_...
STRIPE_WEBHOOK_SECRET=whsec_...
FRONTEND_URL=http://localhost:3000
```

Contenido real de backend/.env.example, usado como plantilla para el .env local.

Paso 4: Aplicar las migraciones

```
alembic upgrade head
```

Paso 5: Levantar el servidor

```
uvicorn main:app --reload --port 8000
```

El backend queda disponible en http://localhost:8000, con documentación interactiva automática en /api/docs (Swagger) y /api/redoc.

4.2. Instalación Local del Frontend

Paso 1: Instalar dependencias

```
cd frontend
npm install
```

Paso 2: Configurar variables de entorno

Copie frontend/.env.local.example a frontend/.env.local:

```
# URL del backend FastAPI
# En desarrollo local: http://localhost:8000/api
# En producción: https://tu-servicio.onrender.com/api
NEXT_PUBLIC_API_URL=http://localhost:8000/api

# Servidor TURN para la videollamada (opcional en desarrollo)
# NEXT_PUBLIC_TURN_URL=turn:ejemplo.relay.metered.ca:80,...
# NEXT_PUBLIC_TURN_USERNAME=
# NEXT_PUBLIC_TURN_CREDENTIAL=
```

Paso 3: Levantar el servidor de desarrollo

```
npm run dev
```

El frontend queda disponible en http://localhost:3000.

> ***Nota:*** *Sin un servidor TURN configurado, la videollamada solo funcionará entre dos personas cuyas redes permitan la conexión directa (no funcionará, por ejemplo, si alguna de las dos está detrás de un NAT simétrico o una red corporativa restrictiva).*

5. Arquitectura del Backend

5.1. Estructura de Carpetas

```
backend/
  core/                  Configuración, base de datos, caché, bus de eventos
    config.py              Settings (pydantic-settings)
    database.py            Motor async de SQLAlchemy y get_db()
    cache.py               Conexión y helpers de Redis
    event_bus.py           Bus de eventos interno (pub/sub en memoria)
    middleware.py          SecurityHeadersMiddleware
    rate_limiter.py        Límite de peticiones (p. ej. en /auth)
    ws_auth.py             Autenticación de WebSocket via query param
  modules/               Nueve módulos de dominio
    identity/            Registro, login, perfil, JWT
    matchmaking/           Cola de emparejamiento por afinidad
    sessions/            Sesión de enfoque, temporizador, WebSocket de sala
    gamification/          XP, nivel, racha, FocusCoins, tienda, confianza
    social/              Amistades, cuota diaria, jardín
    groups/              Grupos, sala de espera, código de invitación
    scheduling/            Citas (sesiones agendadas)
    achievements/          Logros
    payments/            Compra de FocusCoins con Stripe
    reports/             Buzón: reportes y feedback
    announcements/           Anuncios / banners
    admin/               Panel de administración
  alembic/versions/      22 migraciones versionadas
  tests/unit/            Suite de pruebas (pytest)
  main.py                Punto de entrada: registra routers y EventBus
  railway.toml           Configuración de despliegue en Railway
```

5.2. El Patrón de Cuatro Capas por Módulo

Cada módulo de dominio en backend/modules/ sigue, de forma consistente, el mismo patrón de cuatro capas, de abajo hacia arriba:

- models.py — las tablas de SQLAlchemy (clases que heredan de Base).
- repositories/ — acceso a datos puro: consultas SQL vía SQLAlchemy, sin lógica de negocio.
- services/ — la lógica de negocio: valida, orquesta repositorios, publica eventos en el EventBus.
- api/router.py — los endpoints HTTP (FastAPI APIRouter), que traducen peticiones HTTP a llamadas al servicio.

A continuación se muestra el módulo payments completo como ejemplo de las cuatro capas, por ser el más reciente del proyecto y el más ilustrativo del patrón (algunos módulos más antiguos, como gamification, agrupan la capa de repositorio y servicio de forma ligeramente distinta, pero mantienen la misma separación de responsabilidades).

5.2.1. Capa de catálogo/configuración fija — payments/catalog.py

```
"""
Catalogo fijo de paquetes de FocusCoins comprables con dinero real.
"""
from dataclasses import dataclass



@dataclass(frozen=True)
class CoinPack:
```

```
    id: str
    nombre: str
    coins: int
    precio_centavos: int   # en USD, centavos (lo que pide Stripe)


COIN_PACKS: dict[str, CoinPack] = {
    p.id: p
    for p in [
        CoinPack(id="pack_chico",  nombre="Paquete chico",  coins=500,  precio_centavos=299),
        CoinPack(id="pack_medio",  nombre="Paquete medio",  coins=1200, precio_centavos=599),
        CoinPack(id="pack_grande", nombre="Paquete grande", coins=3000,
precio_centavos=1299),
    ]
}
```

backend/modules/payments/catalog.py — fragmento real del repositorio.

5.2.2. Capa de modelo — payments/models.py

```
class CoinPurchase(Base):
    """
    Un intento de compra de FocusCoins con dinero real.

    Se crea en "pendiente" cuando el usuario pide el Checkout de Stripe, y
    pasa a "pagado" solo cuando llega el webhook de Stripe confirmando el
    cobro. stripe_session_id es unico: si Stripe reintenta el mismo
    webhook, el segundo intento no vuelve a acreditar.
    """
    __tablename__ = "coin_purchases"

    id: Mapped[str] = mapped_column(String(36), primary_key=True,
                                    default=lambda: str(uuid.uuid4()))
    user_id: Mapped[str] = mapped_column(
        String(36), ForeignKey("users.id", ondelete="CASCADE"),
        nullable=False, index=True)
    pack_id:          Mapped[str] = mapped_column(String(40), nullable=False)
    coins:            Mapped[int] = mapped_column(Integer, nullable=False)
    precio_centavos:    Mapped[int] = mapped_column(Integer, nullable=False)
    stripe_session_id:  Mapped[str] = mapped_column(String(255), unique=True, nullable=False)
    estado: Mapped[str] = mapped_column(String(20), nullable=False, default="pendiente")
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
server_default=func.now())
    paid_at:    Mapped[datetime | None] = mapped_column(DateTime(timezone=True),
nullable=True)
```

backend/modules/payments/models.py — fragmento real del repositorio.

5.2.3. Capa de servicio — payments/service.py

```
class PaymentsService:
    def __init__(self, db: AsyncSession) -> None:
        self.db = db
        self.gamification_repo = GamificationRepository(db)

    async def crear_checkout(self, user_id: str, user_email: str, pack_id: str) -> str:
        self._requiere_stripe_configurado()
        pack = obtener_pack(pack_id)
        if not pack:
            raise HTTPException(status_code=404, detail="Paquete no encontrado.")
```

```
        session = stripe.checkout.Session.create(
            mode="payment",
            payment_method_types=["card"],
            customer_email=user_email,
            line_items=[{
                "price_data": {
                    "currency": "usd",
                    "unit_amount": pack.precio_centavos,
                    "product_data": {"name": f"SINKA — {pack.nombre} ({pack.coins} FocusCoins)"},
                },
                "quantity": 1,
            }],
            metadata={"user_id": user_id, "pack_id": pack.id},
            success_url=f"{settings.frontend_url}/shop?pago=exito",
            cancel_url=f"{settings.frontend_url}/shop?pago=cancelado",
        )

        compra = CoinPurchase(user_id=user_id, pack_id=pack.id, coins=pack.coins,
                              precio_centavos=pack.precio_centavos,
                              stripe_session_id=session.id, estado="pendiente")
        self.db.add(compra)
        await self.db.commit()
        return session.url

    async def procesar_webhook(self, payload: bytes, firma: str) -> None:
        # ... valida la firma de Stripe, busca la compra por stripe_session_id
        # y solo entonces acredita los coins (ver codigo completo en el repo) ...
        compra.estado = "pagado"
        stats = await self.gamification_repo.get_or_create(compra.user_id)
        stats.focus_coins += compra.coins
        await self.gamification_repo.save(stats)
        await self.db.commit()
```

backend/modules/payments/service.py — fragmento real (abreviado) del repositorio.

5.2.4. Capa de API — payments/api/router.py

```
router = APIRouter(prefix="/payments", tags=["payments"])



@router.get("/packs", response_model=list[CoinPackResponse])
async def listar_paquetes() -> list[CoinPackResponse]:
    return [CoinPackResponse(id=p.id, nombre=p.nombre, coins=p.coins,
                             precio_centavos=p.precio_centavos)
            for p in COIN_PACKS.values()]



@router.post("/checkout", response_model=CheckoutResponse)
async def crear_checkout(
    body: CrearCheckoutRequest,
    current_user: UserResponse = Depends(get_current_user),
    service: PaymentsService = Depends(_service),
) -> CheckoutResponse:
    url = await service.crear_checkout(current_user.id, current_user.email, body.pack_id)
    return CheckoutResponse(checkout_url=url)



@router.post("/webhook")
async def webhook_stripe(request: Request, service: PaymentsService = Depends(_service)) ->
dict:
    """Sin Depends(get_current_user): quien llama esto es Stripe, no un
    usuario de SINKA. La autenticidad se valida con la firma del header
```

```
    Stripe-Signature contra STRIPE_WEBHOOK_SECRET."""
    payload = await request.body()
    firma = request.headers.get("stripe-signature", "")
    await service.procesar_webhook(payload, firma)
    return {"received": True}
```

backend/modules/payments/api/router.py — fragmento real del repositorio.

5.3. Segundo Patrón de Ejemplo: gamification

El módulo gamification es uno de los más antiguos del proyecto y muestra una variante del mismo patrón: separa repositories/ en dos repositorios especializados (gamification_repository.py para XP/racha/FocusCoins y shop_repository.py para la tienda), y concentra servicios adicionales como trust_service.py para el nivel de confianza. El modelo central es UserStats, una fila por usuario con todo su progreso:

```
class UserStats(Base):
    """Estadísticas de gamificación por usuario. Una fila por usuario,
    upsert en cada sesión completada."""
    __tablename__ = "user_stats"

    id: Mapped[str] = mapped_column(String(36), primary_key=True,
                                    default=lambda: str(uuid.uuid4()))
    user_id: Mapped[str] = mapped_column(String(36),
        ForeignKey("users.id", ondelete="CASCADE"), nullable=False, unique=True, index=True)
    xp_total: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    level: Mapped[int] = mapped_column(Integer, nullable=False, default=1)
    streak_current: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    streak_max: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    sessions_completed: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    pomodoros_completed: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    focus_coins: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    # Confianza: baja al abandonar sesiones, sube al completarlas. Solo
    # afecta la prioridad en el emparejamiento, nunca el acceso.
    trust_score: Mapped[int] = mapped_column(Integer, nullable=False, default=100)
    sessions_abandoned: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    last_session_date: Mapped[date | None] = mapped_column(Date, nullable=True)
```

backend/modules/gamification/models.py — fragmento real del repositorio.

5.4. El Bus de Eventos Interno (EventBus)

Para que los módulos no dependan directamente unos de otros, SINKA usa un bus de eventos en memoria: un módulo publica un evento con un nombre y un payload, y cualquier otro módulo puede suscribirse a ese nombre sin que el que publica sepa quién lo escucha. Esto es lo que permite, por ejemplo, que terminar una sesión de enfoque (módulo sessions) dispare a la vez la actualización de XP (gamification), el riego de la planta (social) y la evaluación de logros (achievements), sin que sessions importe código de ninguno de los otros tres.

```
class InternalEventBus:
    def __init__(self) -> None:
        self._subscribers: dict[str, list[Callable]] = defaultdict(list)

    def subscribe(self, event: str, handler: Callable) -> None:
        self._subscribers[event].append(handler)
```

```
    async def publish(self, event: str, payload: Any) -> None:
        handlers = self._subscribers.get(event, [])
        if not handlers:
            return
        results = await asyncio.gather(
            *[handler(payload) for handler in handlers],
            return_exceptions=True,
        )
        for handler, result in zip(handlers, results):
            if isinstance(result, Exception):
                logger.error("EventBus: handler '%s' fallo en evento '%s': %s",
                             handler.__name__, event, result, exc_info=result)


EventBus = InternalEventBus()
```

backend/core/event_bus.py — íntegro, tal como está en el repositorio.

Las suscripciones se registran una sola vez, al arrancar la aplicación, en el evento startup de main.py:

```
@app.on_event("startup")
async def startup_event() -> None:
    # Matchmaking -> Sessions: cuando se crea un match, inicializar la sesion
    EventBus.subscribe("match.created", session_service.on_match_created)
    # Sessions -> Gamification: cuando termina una sesion, actualizar XP y racha
    EventBus.subscribe("session.completed", gamification_service.on_session_completed)
    # Sessions -> Social: si los dos son amigos, la sesion riega su planta
    EventBus.subscribe("session.completed", _regar_jardin)
    # Sessions -> Logros: revisa si la sesion recien completada desbloquea algo
    EventBus.subscribe("session.completed", achievements_service.on_session_completed)
    EventBus.subscribe("group.session.started", achievements_service.on_group_session_started)
    EventBus.subscribe("group_session.completed",
gamification_service.on_group_session_completed)
```

backend/main.py — fragmento real de la función startup_event().

5.5. Autenticación de WebSocket

Los WebSocket no pueden enviar cabeceras HTTP personalizadas en el handshake de conexión, así que el token JWT de acceso se pasa como parámetro de consulta en la URL (?token=...), y se valida con una dependencia dedicada:

```
async def get_current_user_ws(
    websocket: WebSocket,
    token: str = Query(..., description="JWT access token"),
    service: IdentityService = Depends(_get_identity_service),
) -> UserResponse:
    """Extrae y valida el JWT desde el query param ?token=...
    Lanza WebSocketException(4001) si el token es invalido."""
    try:
        return await service.get_current_user(token)
    except Exception:
        raise WebSocketException(
            code=status.WS_1008_POLICY_VIOLATION,
            reason="Token invalido o expirado.",
        )
```

backend/core/ws_auth.py — íntegro, tal como está en el repositorio.

5.6. Configuración y Conexión a la Base de Datos

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")

    app_env: str = "development"
    app_title: str = "SINKA API"
    app_version: str = "1.0.0"
    cors_origins: str = "http://localhost:3000"

    database_url: str
    redis_url: str

    jwt_secret: str
    jwt_algorithm: str = "HS256"
    jwt_access_token_expire_minutes: int = 30
    jwt_refresh_token_expire_days: int = 7

    stripe_secret_key:      str = ""
    stripe_publishable_key: str = ""
    stripe_webhook_secret:  str = ""
    frontend_url: str = "http://localhost:3000"

    @property
    def cors_origins_list(self) -> list[str]:
        return [origin.strip() for origin in self.cors_origins.split(",")]


settings = Settings()
```

backend/core/config.py — íntegro, tal como está en el repositorio.

```
engine = create_async_engine(
    settings.database_url,
    # El "session pooler" de Supabase solo admite 15 conexiones
    # simultaneas EN TOTAL. Se deja margen debajo del limite real para
    # que la app nunca se ahogue a si misma.
    pool_size=8,
    max_overflow=4,
    pool_timeout=10,
    pool_recycle=300,
    echo=settings.app_env == "development",
    connect_args={"ssl": "require", "statement_cache_size": 0},
)

AsyncSessionLocal = async_sessionmaker(bind=engine, class_=AsyncSession,
expire_on_commit=False)


class Base(DeclarativeBase):
    pass


async def get_db() -> AsyncSession:
    async with AsyncSessionLocal() as session:
        yield session
```

backend/core/database.py — íntegro, tal como está en el repositorio.

> ***Nota:*** *El comentario del código explica una decisión real de producción: el pool_size se bajó deliberadamente para no agotar el límite de conexiones del pooler de Supabase, un problema que causó errores de conexión intermitentes en producción antes de este ajuste.*

6. Arquitectura del Frontend

6.1. Estructura de Carpetas

```
frontend/
  src/
    app/                  Quince rutas (App Router de Next.js)
      (auth)/              Grupo de rutas: login, register
      admin/                 Panel de administración
      citas/               Citas agendadas
      dashboard/               Pantalla principal ("Enfocarme")
      grupos/                Grupos
      jardin/               Jardín de plantas
      leaderboard/               Tabla de posiciones
      logros/                Logros
      onboarding/                Asistente inicial tras el registro
      perfil/               Perfil propio y perfil público ([userId])
      session/                Sala de sesión (acompañada) y modo solo
      shop/                 Tienda
      vinculos/               Amigos
      layout.tsx            Layout raíz: fuentes, ThemeProvider, I18nProvider
      globals.css             Variables CSS del sistema de diseño
    components/             NavBar, Notificaciones, ThemeProvider, I18nProvider, ui/
    lib/
      api.ts                Cliente HTTP central (fetch + manejo de tokens)
      theme.ts                Tokens de diseño (color, radius, shadow, fuentes)
      i18n.ts               Diccionario es/en
      webrtc-config.ts          Servidores ICE/STUN/TURN
      peer-connection.ts          Hook de WebRTC para sesión 1 a 1
      group-peer-connections.ts  Variante para sesiones de grupo (N conexiones)
      focus-detector.ts         Detección de inactividad en pantalla
    store/
      auth.store.ts           Estado de autenticación (Zustand)
```

6.2. Manejo de Estado con Zustand

El estado de autenticación (usuario actual, tokens de acceso y refresco) vive en un store de Zustand (frontend/src/store/auth.store.ts), persistido en el navegador. Los componentes que necesitan saber si hay una sesión activa, o el token para llamar a la API, usan el hook useAuthStore() en vez de pasar esos datos por props a través de todo el árbol de componentes.

6.3. El Cliente de API (lib/api.ts)

Toda llamada HTTP del frontend hacia el backend pasa por una función central request<T>(), que añade las cabeceras necesarias, intercepta respuestas 401 para renovar el token de acceso automáticamente usando el token de refresco, y traduce los errores de validación de FastAPI (que llegan como una lista de objetos) a un mensaje legible en español:

```
function mensajeDeError(body: unknown, status: number): string {
  const detalle = (body as { detail?: unknown } | null)?.detail;
  if (typeof detalle === "string" && detalle.trim()) return detalle;
```

```
  if (Array.isArray(detalle)) {
    // FastAPI devuelve una lista de errores de validacion, uno por campo.
    // Mostrarla tal cual daba el famoso "[object Object]" en pantalla.
    const NOMBRES: Record<string, string> = {
      alias: "el nombre visible", email: "el correo",
      username: "el nombre de usuario", password: "la contraseña", /* ... */
    };
    const frases = detalle.map((e) => {
      const item = e as { msg?: string; loc?: unknown[] };
      const motivo = (item.msg ?? "").replace(/^\w+ error,\s*/i, "").trim();
      const campo = Array.isArray(item.loc) ? String(item.loc[item.loc.length - 1]) : "";
      const legible = NOMBRES[campo];
      return legible ? `${motivo} (${legible})` : motivo;
    }).filter(Boolean);
    if (frases.length) return frases.join(" ");
  }
  if (status === 401) return "Tu sesión expiró. Vuelve a iniciar sesión.";
  if (status === 403) return "No tienes permiso para hacer eso.";
  if (status === 404) return "No encontramos lo que buscabas.";
  if (status === 429) return "Demasiados intentos. Espera un momento.";
  if (status >= 500)  return "El servidor tuvo un problema. Intenta de nuevo.";
  return "Algo salió mal.";
}
```

frontend/src/lib/api.ts — fragmento real del repositorio.

```
async function request<T>(path: string, options?: RequestInit, reintentado = false): Promise<T> {
  const res = await fetch(`${API_BASE}${path}`, {
    ...options,
    headers: { "Content-Type": "application/json", ...options?.headers },
  });

  if (!res.ok) {
    // Token caducado: se renueva una vez y se repite la peticion original
    const llevaToken = Boolean((options?.headers as Record<string, string> |
undefined)?.Authorization);
    if (res.status === 401 && llevaToken && !reintentado && path !== "/auth/refresh") {
      const nuevo = await renovarAcceso();
      if (nuevo) {
        return request<T>(path, { ...options, headers: conNuevoToken(options?.headers, nuevo) },
true);
      }
    }
    const body = await res.json().catch(() => ({ detail: "Error desconocido" }));
    throw new ApiError(mensajeDeError(body, res.status), res.status);
  }
  // ... continúa con el parseo de la respuesta exitosa ...
}
```

frontend/src/lib/api.ts — renovación automática del token de acceso.

6.4. Internacionalización (i18n)

SINKA usa un diccionario propio en frontend/src/lib/i18n.ts (claves en español e inglés) junto con I18nProvider (React Context) y el hook useTranslation(), en vez de una librería externa de i18n. Cada texto traducible referencia una clave (por ejemplo, topic.software) en vez de escribirse literalmente en el componente.

6.5. Tema Claro/Oscuro (ThemeProvider)

Los tokens de diseño (colores, radios, sombras, tipografías) están centralizados en frontend/src/lib/theme.ts y expuestos como variables CSS, lo que permite cambiar de tema claro a oscuro de forma dinámica sin recompilar. El ThemeProvider persiste la preferencia en localStorage y, si el usuario inició sesión, también la guarda en el backend para que se mantenga entre dispositivos.

6.6. Ejemplo de Página: Dashboard

frontend/src/app/dashboard/page.tsx es la pantalla principal. Declara el catálogo de áreas de trabajo (usado tanto en la búsqueda de pareja como en los filtros de Grupos) y gestiona el estado de la búsqueda de pareja mediante una máquina de estados simple (config → searching → matched → timeout → error):

```
const TOPICS: { value: string; clave: ClaveTraduccion; Icon: ComponentType<LucideProps> }[] = [
  { value: "software",  clave: "topic.software",  Icon: Code },
  { value: "mobile",    clave: "topic.mobile",    Icon: Smartphone },
  { value: "web",       clave: "topic.web",       Icon: Globe },
  { value: "design",    clave: "topic.design",    Icon: Palette },
  { value: "video",     clave: "topic.video",     Icon: Clapperboard },
  { value: "writing",   clave: "topic.writing",   Icon: Pencil },
  // ... resto de áreas (dibujo, música, marketing, fotografía, datos, idiomas, otro)
];

type Screen = "config" | "searching" | "matched" | "timeout" | "error";

export default function DashboardPage() {
  const router = useRouter();
  const { user, accessToken: token, hidratado } = useAuthStore();
  const { t } = useTranslation();

  const [topic, setTopic] = useState("");
  const [taskTitle, setTaskTitle] = useState("");
  const [screen, setScreen] = useState<Screen>("config");
  const wsRef = useRef<WebSocket | null>(null);
  const [stats, setStats] = useState<UserStats | null>(null);
  // ...
}
```

frontend/src/app/dashboard/page.tsx — fragmento real del repositorio.

7. Base de Datos

7.1. Tablas Principales

El esquema se construye de forma incremental mediante 22 migraciones Alembic, desde la creación de users y user_sessions hasta la última, que añade la categoría de banners de perfil a la tienda. Las tablas más relevantes son:

Tabla	Módulo	Contenido
users	identity	Cuenta: correo, nombre de usuario, contraseña cifrada, rol, perfil público

Tabla	Módulo	Contenido
user_sessions	identity	Tokens de refresco activos, con fecha de expiración y revocación
matches / focus_sessions	matchmaking / sessions	Emparejamientos y sesiones de enfoque, con su estado y duración
user_stats	gamification	XP, nivel, racha, FocusCoins, confianza por usuario (una fila por usuario)
shop_items / user_inventory	gamification	Catálogo de cosméticos y lo que cada usuario ha comprado
session_penalties	gamification	Historial de penalizaciones por abandono de sesión
friendships	social	Amistades y solicitudes entre usuarios
groups / group_members	groups	Grupos de trabajo por área y sus integrantes
appointments	scheduling	Citas agendadas entre dos usuarios
achievements / user_achievements	achievements	Catálogo de logros y los desbloqueados por cada usuario
coin_purchases	payments	Compras de FocusCoins con dinero real, vía Stripe
reports	reports	Reportes de usuarios y mensajes de feedback general (buzón)
announcements	announcements	Anuncios/banners mostrados en el Dashboard
admin_settings / admin_audit_log	admin	Parámetros configurables y registro de auditoría

7.2. Migraciones con Alembic

Cada cambio de esquema se versiona como un archivo independiente en backend/alembic/versions/, numerado secuencialmente (0001 a 0022). Para aplicar las migraciones pendientes:

```
cd backend
alembic upgrade head
```

Para crear una migración nueva tras modificar un archivo models.py:

```
alembic revision -m "descripcion_breve"
# completar upgrade() y downgrade() a mano, o con --autogenerate
```

7.2.1. Ejemplo: migración 0001 (tablas base)

```
def upgrade() -> None:
    op.create_table(
        "users",
        sa.Column("id", sa.String(36), primary_key=True),
        sa.Column("email", sa.String(255), nullable=False, unique=True),
        sa.Column("username", sa.String(50), nullable=False, unique=True),
```

```
        sa.Column("hashed_password", sa.String(255), nullable=False),
        sa.Column("is_active", sa.Boolean(), nullable=False, server_default="true"),
        sa.Column("created_at", sa.DateTime(timezone=True), server_default=sa.func.now()),
        sa.Column("updated_at", sa.DateTime(timezone=True), server_default=sa.func.now()),
    )
    op.create_index("ix_users_email", "users", ["email"])
    op.create_index("ix_users_username", "users", ["username"])

    op.create_table(
        "user_sessions",
        sa.Column("id", sa.String(36), primary_key=True),
        sa.Column("user_id", sa.String(36), sa.ForeignKey("users.id", ondelete="CASCADE"),
nullable=False),
        sa.Column("refresh_token", sa.String(512), nullable=False, unique=True),
        sa.Column("is_revoked", sa.Boolean(), nullable=False, server_default="false"),
        sa.Column("created_at", sa.DateTime(timezone=True), server_default=sa.func.now()),
        sa.Column("expires_at", sa.DateTime(timezone=True), nullable=False),
    )
```

backend/alembic/versions/0001_create_users_and_sessions.py — fragmento real del repositorio.

7.2.2. Ejemplo: migración 0022 (datos semilla)

No todas las migraciones crean tablas: algunas insertan datos semilla, como la 0022, que agrega seis banners de perfil comprables a la tienda:

```
IDS = (
    "item-banner-amanecer", "item-banner-bosque", "item-banner-oceano",
    "item-banner-lavanda", "item-banner-fuego", "item-banner-oro",
)

def upgrade() -> None:
    op.execute("""
        INSERT INTO shop_items (id, name, description, category, price_fc, preview, is_active) VALUES
        ('item-banner-amanecer', 'Amanecer', 'Banner de perfil degradado amanecer',
'profile_banner', 30,
         'linear-gradient(135deg, #ff9a76, #ffd36e)', true),
        ('item-banner-oceano',   'Océano',   'Banner de perfil degradado océano',   'profile_banner', 40,
         'linear-gradient(135deg, #1b3a57, #4fa8d8)', true)
        -- ... (seis banners en total) ...
    """)

def downgrade() -> None:
    ids_sql = ", ".join(f"'{i}'" for i in IDS)
    op.execute(f"DELETE FROM user_inventory WHERE item_id IN ({ids_sql})")
    op.execute(f"UPDATE users SET equipped_banner_item_id = NULL WHERE
equipped_banner_item_id IN ({ids_sql})")
    op.execute(f"DELETE FROM shop_items WHERE id IN ({ids_sql})")
```

backend/alembic/versions/0022_shop_banners.py — fragmento real (abreviado) del repositorio.

8. Módulos del Sistema

A continuación se describe brevemente cada módulo de dominio del backend y un endpoint representativo de su API.

8.1. identity — Identidad y Autenticación

Registro, inicio de sesión, renovación de tokens, edición de perfil propio y consulta de perfiles públicos. Emite tokens JWT de acceso (30 minutos) y de refresco (7 días).

```
POST /api/auth/register       -> TokenResponse (201)
POST /api/auth/login          -> TokenResponse
GET  /api/users/{user_id}/profile  -> PublicProfileFull
```

8.2. matchmaking — Emparejamiento

Gestiona la cola de personas buscando pareja, con afinidad por área de trabajo y respaldo general tras 30 segundos de espera.

```
GET /api/matchmaking/status  -> estado actual de la búsqueda del usuario
```

8.3. sessions — Sesiones de Enfoque

El temporizador Pomodoro compartido, el canal WebSocket de la sala (temporizador, chat, señalización WebRTC) y el ciclo de vida de una sesión de enfoque.

```
GET /api/sessions/active           -> FocusSessionResponse | null
GET /api/sessions/{session_id}     -> FocusSessionResponse
```

8.4. gamification — XP, Nivel, Racha y Tienda

Calcula experiencia y nivel al completar sesiones y pomodoros, mantiene la racha diaria, administra FocusCoins, el nivel de confianza y la tienda de cosméticos (sub-router shop_router.py).

```
GET  /api/gamification/stats       -> UserStatsResponse
GET  /api/gamification/leaderboard  -> list[LeaderboardEntry]
POST /api/shop/buy/{item_id}       -> BuyItemResponse
```

8.5. social — Vínculos y Jardín

Solicitudes de amistad (con cuota diaria), bloqueo de usuarios y el Jardín: el crecimiento de la planta asociada a cada vínculo.

```
POST /api/friends/request/{user_id}  -> 201
GET  /api/friends                    -> list[FriendResponse]
```

8.6. groups — Grupos

Grupos de trabajo por área, con sala de espera (lobby), código de invitación y transferencia de propiedad del grupo.

```
GET  /api/groups/explore          -> list[GroupResponse]
POST /api/groups/{group_id}/code  -> InviteCodeResponse
POST /api/groups/join             -> GroupResponse  (por código)
```

8.7. scheduling — Citas

Sesiones de enfoque agendadas para una hora concreta entre dos amigos, con invitación, aceptación/rechazo y unión directa a la hora pactada.

```
POST /api/appointments              -> AppointmentResponse (201)
POST /api/appointments/{id}/accept   -> AppointmentResponse
POST /api/appointments/{id}/join     -> unión a la sesión agendada
```

8.8. achievements — Logros

Catálogo de logros agrupados por categoría, evaluados automáticamente cuando el EventBus notifica eventos como session.completed.

```
GET /api/achievements  -> AchievementsSummary
```

8.9. payments — Pagos

Descrito en detalle en la sección 5.2 como ejemplo principal del patrón de cuatro capas: catálogo de paquetes, creación de la sesión de Stripe Checkout y procesamiento del webhook de confirmación.

8.10. reports — Buzón

Reportes de usuarios sobre otros usuarios y feedback general, consumidos y respondidos desde el panel de administración.

```
POST /api/reports          -> ReportResponse
POST /api/feedback          -> ReportResponse
GET  /api/reports/mine      -> list[ReportResponse]
```

8.11. announcements — Anuncios

Banners informativos mostrados en el Dashboard, con fecha de inicio y expiración opcionales.

```
GET /api/announcements/active  -> AnnouncementPublic | null
```

8.12. admin — Panel de Administración

Agrupa las operaciones administrativas de todos los módulos anteriores (usuarios, tienda, reportes, grupos, anuncios, parámetros y auditoría) descritas en detalle en el Manual de Administrador.

```
GET  /api/admin/stats               -> AdminStats
GET  /api/admin/users               -> AdminUserList
POST /api/admin/users/{id}/ban      -> AdminUserRow
PATCH /api/admin/users/{id}/role    -> AdminUserRow
GET  /api/admin/audit               -> list[AdminAuditRow]
```

9. API REST

9.1. Formato de Respuesta y Autenticación

Todos los endpoints responden en JSON. Los que requieren autenticación esperan la cabecera Authorization: Bearer <access_token>, validada mediante la dependencia get_current_user de FastAPI. Cuando el token expira, el backend responde 401 y el cliente (frontend/src/lib/api.ts) renueva el token automáticamente con el token de refresco antes de reintentar la petición una sola vez.

9.2. Endpoints Principales por Módulo

Módulo	Prefijo	Endpoints representativos
identity	/api/auth, /api/users	register, login, refresh, me, users/{id}/profile
matchmaking	/api/matchmaking	status
sessions	/api/sessions	active, {id}, WebSocket de sala
gamification	/api/gamification	stats, solo/complete, leaderboard, trust, presence
shop (gamification)	/api/shop	items, buy/{id}, inventory, equip/{id}, unequip-banner
social	/api/friends	(listar), requests, request/{id}, accept/{id}, block/{id}
groups	/api/groups	explore, mine, {id}, {id}/code, join, {id}/lobby/state
scheduling	/api/appointments	(crear/listar), invitations, {id}/accept, {id}/join
achievements	/api/achievements	(resumen)
payments	/api/payments	packs, checkout, webhook
reports	/api/reports, /api/feedback	reports, feedback, reports/mine
announcements	/api/announcements	active
admin	/api/admin	stats, users, users/{id}/ban, users/ban-lote, users/{id}/role, shop, reports, groups, revenue, announcements, settings, audit

Nota: *Esta tabla resume los grupos de endpoints, no la lista exhaustiva de cada uno. La documentación interactiva completa y siempre actualizada está disponible en /api/docs (Swagger UI) mientras el backend está corriendo.*

10. WebSockets

10.1. El Canal de la Sesión

Cuando dos usuarios son emparejados, ambos navegadores abren una conexión WebSocket hacia el backend (autenticada con el JWT como query param, según se explicó en la sección 5.5). Por ese canal viajan tres tipos de mensajes: la sincronización del temporizador Pomodoro compartido, los mensajes de chat durante los descansos, y los mensajes de señalización WebRTC necesarios para establecer la videollamada.

10.2. Señalización WebRTC

SINKA no transporta video ni audio a través de su propio servidor: el backend solo retransmite los mensajes de señalización (ofertas y respuestas SDP, candidatos ICE) entre los dos participantes de la sala, usando el mismo canal WebSocket de la sesión con un tipo de mensaje "signal". Una vez que ambos navegadores completan el intercambio, el video y el audio viajan directamente entre ellos (peer-to-peer), apoyados por los servidores ICE/STUN/TURN definidos en frontend/src/lib/webrtc-config.ts.

Para sesiones de grupo, donde puede haber más de dos participantes, el frontend usa una variante en group-peer-connections.ts que mantiene una conexión WebRTC independiente por cada par de participantes conectados simultáneamente.

11. Despliegue

11.1. Backend en Railway

El backend se despliega en Railway a partir del repositorio de GitHub. La configuración de build y arranque está en backend/railway.toml:

```
[build]
builder = "nixpacks"

[deploy]
startCommand = "uvicorn main:app --host 0.0.0.0 --port $PORT"
healthcheckPath = "/api/health"
healthcheckTimeout = 60
restartPolicyType = "on_failure"
restartPolicyMaxRetries = 3
```

backend/railway.toml — íntegro, tal como está en el repositorio.

Railway inyecta automáticamente la variable $PORT; el resto de variables de entorno (DATABASE_URL, REDIS_URL, JWT_SECRET, claves de Stripe, CORS_ORIGINS, FRONTEND_URL) se configuran manualmente en el panel de Railway, con los mismos nombres que en backend/.env.example pero apuntando a los servicios de producción (Supabase y Upstash). El endpoint /api/health, usado por el healthcheck de Railway, verifica la conexión a PostgreSQL y a Redis antes de reportar el servicio como saludable.

11.2. Frontend en Vercel

El frontend no requiere un archivo de configuración adicional (no existe vercel.json en el repositorio): Vercel detecta automáticamente que es un proyecto Next.js y aplica su build estándar (next build). La única configuración necesaria se hace desde el dashboard de Vercel, en Project Settings → Environment Variables, replicando las claves de frontend/.env.local.example con los valores de producción:

- NEXT_PUBLIC_API_URL = https://sinka-production-f624.up.railway.app/api
- NEXT_PUBLIC_TURN_URL, NEXT_PUBLIC_TURN_USERNAME, NEXT_PUBLIC_TURN_CREDENTIAL (servidor TURN de producción, para que la videollamada funcione detrás de redes restrictivas).

Cada push a la rama principal del repositorio dispara automáticamente un nuevo despliegue en Vercel.

11.3. Webhook de Stripe en Producción

Para que las compras de FocusCoins se acrediten en producción, es necesario registrar el endpoint de webhook en el dashboard de Stripe (Developers → Webhooks), apuntando a https://sinka-production-f624.up.railway.app/api/payments/webhook, suscrito al evento checkout.session.completed. El secreto que Stripe genera al crear el webhook se copia a la variable STRIPE_WEBHOOK_SECRET en Railway.

> ***Nota:*** *Usar las claves sk_test_ / pk_test_ / whsec_ mientras se prueba el flujo de pagos, y solo cambiar a las claves sk_live_ / pk_live_ cuando se vaya a cobrar con dinero real.*

12. Extensión y Mantenimiento

12.1. Cómo Agregar una Funcionalidad Nueva

Para mantener la consistencia arquitectónica, cualquier funcionalidad nueva de backend que represente un dominio propio debe seguir el mismo patrón de cuatro capas descrito en la sección 5.2. Por ejemplo, para agregar un módulo nuevo llamado notifications:

Paso 1: Crear la carpeta del módulo

```
backend/modules/notifications/
  __init__.py
  models.py
  repositories/
  services/
  api/
    router.py
```

Paso 2: Definir el modelo

Crear la(s) clase(s) SQLAlchemy en models.py, heredando de Base (core/database.py).

Paso 3: Crear la migración

```
cd backend
alembic revision -m "create_notifications_table"
# completar upgrade()/downgrade() y aplicar con: alembic upgrade head
```

Paso 4: Implementar el repositorio

Las consultas a la base de datos, sin lógica de negocio, en repositories/notifications_repository.py.

Paso 5: Implementar el servicio

La lógica de negocio en services/notifications_service.py. Si el módulo necesita reaccionar a eventos de otros módulos (por ejemplo, notificar cuando alguien acepta una solicitud de amistad), suscribirse al EventBus en vez de importar el módulo social directamente.

Paso 6: Exponer los endpoints

Crear el APIRouter en api/router.py, siguiendo el mismo estilo de los routers existentes (ver payments/api/router.py como referencia).

Paso 7: Registrar el módulo en main.py

```
from modules.notifications.api.router import router as notifications_router
# ...
app.include_router(notifications_router, prefix="/api")
# y si define modelos nuevos:
import modules.notifications.models  # noqa: F401
```

Paso 8: Escribir pruebas unitarias

Agregar pruebas en backend/tests/unit/, siguiendo el patrón de los archivos existentes (por ejemplo test_trust_score.py), y ejecutarlas con pytest antes de hacer commit.

Paso 9: Frontend: consumir el nuevo endpoint

Agregar las funciones correspondientes en frontend/src/lib/api.ts (siguiendo el patrón de los objetos *Api existentes, como gamificationApi o announcementsApi) y construir la pantalla o componente en frontend/src/app/.

12.2. Ejecutar la Suite de Pruebas

```
cd backend
pytest
```

La configuración de pytest está en backend/pytest.ini y backend/tests/conftest.py. Las pruebas cubren, entre otras cosas, el crecimiento del jardín, el emparejamiento por afinidad y con bloqueo, la extensión del Pomodoro y el cálculo del nivel de confianza.

13. Buenas Prácticas del Proyecto

- Respetar las cuatro capas por módulo (models → repositories → services → api). La lógica de negocio va en services/, nunca directamente en el router.
- Comunicar módulos entre sí mediante el EventBus, no con imports cruzados, salvo cuando un módulo necesita explícitamente un repositorio de otro (como payments usando GamificationRepository para acreditar FocusCoins).

- Nunca confiar en datos que el cliente pueda falsificar para operaciones sensibles: los FocusCoins de una compra se acreditan solo al confirmar el webhook de Stripe, nunca al volver el usuario a la página de éxito.
- Variables de entorno siempre fuera del código: nunca commitear un archivo .env ni credenciales reales; usar los .env.example como plantilla documentada.
- Todo endpoint que modifique datos de otro usuario (acciones de administrador) debe verificar el rol en el backend, no solo ocultar el botón en el frontend.
- Mensajes de error traducidos a texto legible para la persona usuaria, nunca mostrar el objeto de error crudo en la interfaz.
- Cuidar el límite de conexiones del pool de base de datos al dimensionar pool_size/max_overflow, especialmente con proveedores gestionados que imponen un límite total bajo (como el pooler de Supabase).
- Cada migración de Alembic debe ser reversible: implementar siempre downgrade() además de upgrade().
- Mantener cubierta con pruebas unitarias la lógica de negocio pura (cálculo de XP, racha, confianza, crecimiento del jardín), que no depende de I/O y es rápida de probar.

14. Diagramas y Recorrido de una Petición

Esta sección complementa en imágenes lo que los capítulos 2 y 5 explican en prosa: la arquitectura general del sistema, qué pasa cuando un módulo publica un evento, y el recorrido completo de una petición real desde el clic en el navegador hasta la respuesta en pantalla. La idea es que alguien que recién llega al proyecto pueda ubicarse de un vistazo antes de entrar al detalle de cada módulo.

14.1. Diagrama de Arquitectura General

El siguiente diagrama resume la tabla de la sección 2.1: qué piezas componen SINKA y cómo se conectan entre sí. El backend es un único proceso FastAPI, pero las flechas hacia la derecha muestran cómo el EventBus lo mantiene desacoplado internamente de sus propios módulos de dominio.

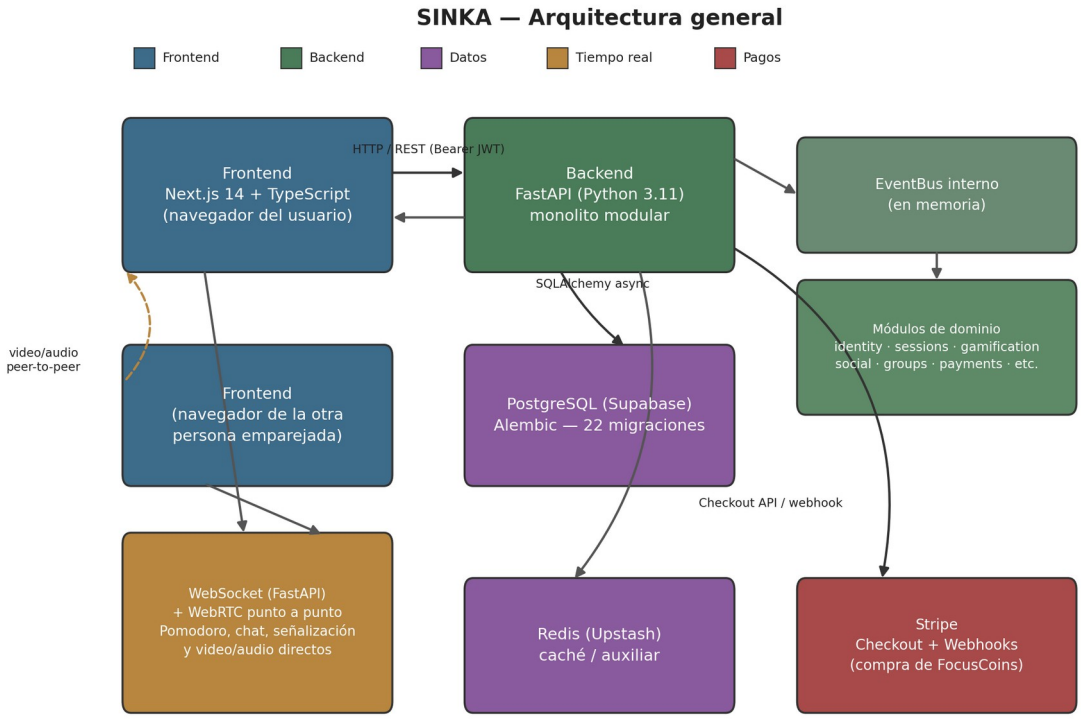


Figura 1. Arquitectura general de SINKA — elaboración propia.

14.2. El EventBus en Acción

Tomando el ejemplo ya descrito en la sección 5.4, el siguiente diagrama muestra qué pasa en el instante en que termina una sesión de enfoque: sessions publica un único evento y tres módulos reaccionan a él en paralelo, sin que sessions sepa que existen. Para agregar un cuarto módulo que también necesite reaccionar a este evento, basta con suscribirlo en startup_event() — no hace falta tocar sessions en absoluto.

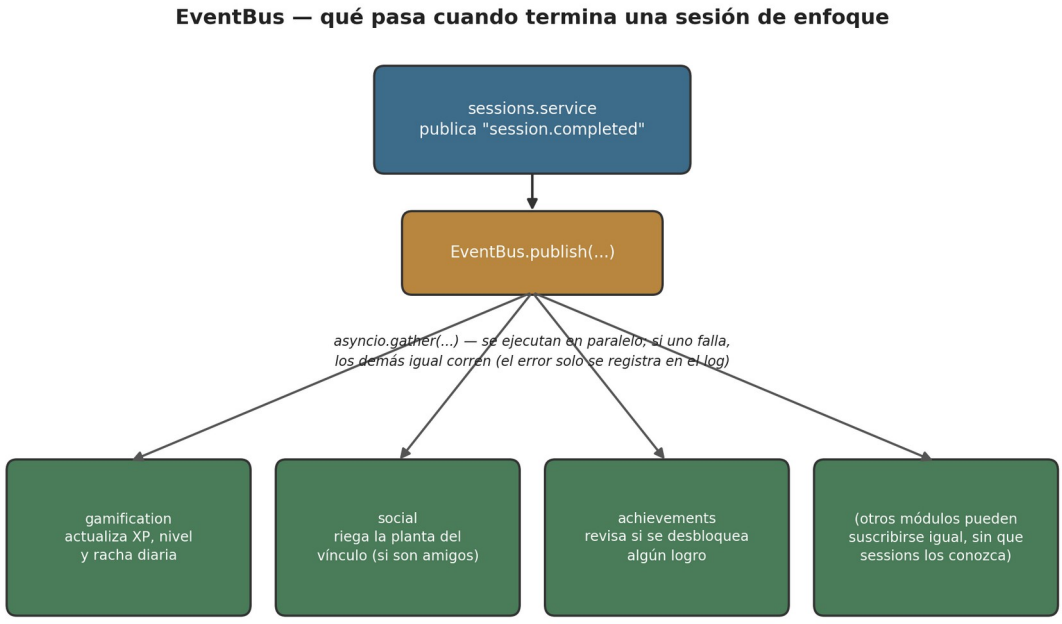


Figura 2. Suscriptores de "session.completed" en el EventBus — elaboración propia.

14.3. Recorrido de una Petición Completa

Entender el patrón de cuatro capas (sección 5.2) en abstracto es un paso; seguir una petición real de principio a fin por las cuatro capas es lo que realmente ayuda a ubicarse en el código antes de modificarlo. El siguiente diagrama sigue el caso más simple posible: comprar un ítem de la tienda con FocusCoins, un endpoint que además no usa una capa de servicio separada (toda la lógica vive directamente en el router), algo que también conviene saber reconocer al leer el código existente.



Figura 3. De clic a respuesta — comprar un ítem de la tienda — elaboración propia.

El mismo recorrido aplica, con más pasos, a cualquier endpoint que sí separe router / service / repository: lo único que cambia es que el paso 4 (reglas de negocio) vive en services/ en vez de en el propio router, y services/ es quien llama a repositories/ para tocar la base de datos.

15. Problemas Comunes al Instalar y Ejecutar en Local

Lista de problemas reales con los que es fácil toparse al levantar el proyecto por primera vez, con su causa típica y cómo resolverlos. No reemplaza leer el mensaje de error completo, pero ahorra las primeras búsquedas.

- **Problema:** uvicorn: "ModuleNotFoundError: No module named 'fastapi'" (o cualquier otro paquete).

Causa: El entorno virtual no está activado, o las dependencias no se instalaron dentro de él.

Solución: Confirmar que el prompt muestre (venv) antes de correr uvicorn. Si no, repetir . \venv\Scripts\Activate.ps1 (o source venv/bin/activate) y luego pip install -r requirements.txt.

- **Problema:** Al arrancar el backend: error de conexión a PostgreSQL ("connection refused" o fallo de SSL).

Causa: DATABASE_URL mal copiada desde Supabase, contraseña con caracteres especiales sin codificar en la URL, o el proyecto de Supabase está pausado (los planes gratuitos se pausan por inactividad).

Solución: Revisar que la URL tenga el prefijo postgresql+asyncpg:// (no postgresql://), que la contraseña esté URL-encoded si tiene @, # o similares, y que el proyecto en el dashboard de Supabase figure como activo.

- **Problema:** El backend arranca pero las operaciones con caché fallan o quedan colgadas.

Causa: Upstash exige TLS: la URL debe usar el esquema rediss:// (con doble "s"), no redis://.

Solución: Verificar que REDIS_URL en .env empiece exactamente con rediss://, tal como viene en el panel de Upstash.

- **Problema:** En el navegador: "has been blocked by CORS policy" al llamar a la API desde el frontend local.

Causa: CORS_ORIGINS en backend/.env no incluye el origen exacto desde el que corre el frontend.

Solución: Agregar http://localhost:3000 (sin barra final) a CORS_ORIGINS y reiniciar uvicorn — los cambios en .env no se recargan solos, ni siquiera con --reload.

- **Problema:** El frontend sigue apuntando a la API vieja después de cambiar NEXT_PUBLIC_API_URL en .env.local.

Causa: Next.js incrusta las variables NEXT_PUBLIC_* en el bundle al iniciar el servidor de desarrollo, no las relee en caliente.

Solución: Detener npm run dev (Ctrl+C) y volver a ejecutarlo después de cualquier cambio en .env.local.

- **Problema:** alembic upgrade head falla con "Multiple heads" o "Target database is not up to date".

Causa: Dos ramas de trabajo crearon migraciones distintas a partir de la misma revisión base, o la base de datos local quedó en una revisión que ya no existe en el historial.

Solución: Correr alembic heads para ver las revisiones en conflicto y alembic merge para unificarlas en una sola cabeza antes de aplicar upgrade head. En un entorno local descartable, también es válido recrear la base de datos desde cero y volver a aplicar todas las migraciones.

- **Problema:** La videollamada nunca conecta entre dos personas (pero sí entre dos pestañas del mismo navegador).

Causa: Sin un servidor TURN configurado (NEXT_PUBLIC_TURN_URL), WebRTC solo logra conectar cuando ambas redes permiten la conexión directa; una NAT simétrica o una red corporativa la bloquean.

Solución: Para desarrollo esto puede ignorarse si se prueba en dos pestañas de la misma máquina. Para replicar el problema real, configurar un servidor TURN de prueba (hay niveles gratuitos, como el de metered.ca) en las variables NEXT_PUBLIC_TURN_*.

- **Problema:** El webhook de Stripe nunca acredita los FocusCoins al comprar un paquete en local.

Causa: Stripe no puede llamar a http://localhost:8000 directamente desde internet: el webhook necesita un reenvío explícito.

Solución: Instalar el Stripe CLI y correr stripe listen --forward-to localhost:8000/api/payments/webhook; el comando imprime un whsec_... temporal que debe copiarse a STRIPE_WEBHOOK_SECRET mientras dura la sesión de prueba.

16. Mapa Mental del Repositorio y Glosario

Una referencia rápida para cuando hay que hacer un cambio puntual y no se quiere releer todo el manual: dónde está cada cosa, y qué significan los términos propios del proyecto que no son estándar de la industria.

16.1. ¿Dónde Busco...?

Quiero...	Lo busco en...
Cambiar cuánta XP da completar una sesión	backend/modules/gamification/service.py (o repositorio equivalente), función que procesa session.completed
Agregar un ítem nuevo a la tienda	Seed de datos de shop_items (migración de Alembic) + backend/modules/gamification/models.py si necesita un campo nuevo
Cambiar el precio o el catálogo de FocusCoins comprables	backend/modules/payments/catalog.py
Agregar un campo al perfil de usuario	backend/modules/identity/models.py, su migración de Alembic, el schema en identity/schemas/auth.py y frontend/src/lib/api.ts (interfaz UserResponse)
Cambiar un texto que ve el usuario en una pantalla	El archivo .tsx de esa pantalla en frontend/src/app/ (o el diccionario de i18n si el texto está traducido)
Agregar un nuevo tipo de notificación/evento interno	Publicar con EventBus.publish(...) donde ocurre el hecho, y suscribir el handler en startup_event() de backend/main.py
Revisar o cambiar una regla de permisos de administrador	backend/modules/admin/ y la dependencia get_current_user / verificación de role en cada router
Entender cómo se guarda y valida el token de sesión	backend/modules/identity/services/identity_service.py (emisión) y core/ws_auth.py o identity/api/dependencies.py (validación)

16.2. Glosario de Términos Propios de SINKA

Término	Significado en SINKA
FocusCoins (FC)	Moneda interna que se gana completando sesiones de enfoque, o se compra con dinero real vía Stripe. Se gasta en la tienda de cosméticos.
Confianza / trust score	Puntaje de 0 a 100 por usuario que baja al abandonar sesiones o grupos sin avisar, y sube con el buen comportamiento; afecta la prioridad en el emparejamiento.
Jardín	Representación visual de cada vínculo de amistad: la planta asociada crece cuando ambos completan sesiones juntos.
Racha (streak)	Días consecutivos en los que el usuario completó al menos una sesión de enfoque.
Lobby	Sala de espera de un grupo antes de que la sesión compartida arranque formalmente.
Respaldo (matchmaking)	Cuando nadie coincide por afinidad de área de trabajo tras 30 segundos, el sistema empareja igual con cualquier persona disponible.
EventBus	Bus de eventos interno en memoria (backend/core/event_bus.py) que desacopla los módulos de dominio entre sí; ver sección 5.4.
Monolito modular	Un único proceso de backend (no microservicios), pero organizado internamente en módulos de dominio independientes que no se importan entre sí directamente.
Señalización (WebRTC)	El intercambio inicial de mensajes SDP/ICE que permite a dos navegadores negociar una conexión de video directa; viaja por el WebSocket de la sesión, no por WebRTC en sí.
Buzón	Módulo unificado de reportes de usuarios y feedback general, consumido y respondido desde el panel de administración.